



STUDY CASE

Tugas Anda mengisi bagian dalam itu

Antrean dilayani dari depan ke belakang, satu per satu, dan tidak pernah mundur. Petugas tidak pernah perlu bertanya "siapa yang ada DI DEPAN orang ini?" yang ia perlukan cuma satu, yaitu "siapa BERIKUTNYA?".

<pre> head [21] -> [47] -> [33] -> [58] -> nullptr ^ ^ dilayani duluan paling belakang </pre>		
head	Alamat orang paling depan .	
next	Alamat orang berikutnya. Orang paling belakang ditandai dengan `next` bernilai `nullptr`, artinya "tidak ada lagi sesudah ini".	
nullptr	Alamat kosong.	
FUNCTION YANG SUDAH DISEDIAKAN & TIDAK DINILAI		
tambahAwal	Menyisipkan pasien prioritas di paling depan	Sisip Depan
panjang	Melihat banyaknya orang yang sedang mengantri	Traversal
cari	Menemukan node seorang pasien	Searching

Satu pagi di Klinik Sehat Selalu.

1. Pukul 07.00 loket buka. Pasien berdatangan, mengambil nomor, dan berdiri di posisi PALING BELAKANG antrian. Satu per satu, sampai ruang tunggu terisi. **(SOAL 1)**
2. Petugas melirik layar untuk tahu ada berapa orang yang sedang menunggu **(FUNGSI panjang)**. Datang juga seorang pasien lansia yang harus didahulukan, dan ia langsung ditempatkan di posisi paling depan **(Fungsi tambahAwal)**.
3. Seorang pasien menghampiri petugas dan bertanya, "Mbak, masih berapa orang lagi di depan saya?" Petugas mengetikkan nomor antriannya, dan sistem harus menjawab berapa orang yang masih mengantri di depannya. **(SOAL 2)**
4. Ada pasien yang membatalkan antreannya karena tidak bisa menunggu lebih lama. Ia harus dikeluarkan dari antrian, dan antriannya tidak boleh putus. Orang di belakangnya harus maju menyambung ke orang di depannya. **(SOAL 3)**
5. Petugas perlu membetulkan nomor seorang pasien yang salah ketik, jadi ia mencari data orang itu lebih dulu **(Fungsi cari)**.
6. Pukul 16.00 jam layanan habis. Locket ditutup dan seluruh sisa antrean dibubarkan sekaligus. **(SOAL 4)**

1. tambahAkhir

30 Poin

Quest	Tempatkan seorang pasien baru di posisi paling belakang antrian. Seluruh pasien yang sudah ada tetap berada di depannya dengan urutan yang sama persis, dan sesudah pasien baru itu tidak ada lagi siapa-siapa.	
Parameter/ Variabel	head	Alamat orang paling depan, milik pemanggil. Bertanda `&` sehingga pergantian orang terdepan ikut terasa oleh pemanggil. Boleh bernilai `nullptr`, artinya antrean sedang kosong
	data	Nomor antrian pasien yang baru datang
	return	`true` bila pasien baru berhasil masuk antrian
Gambaran	Sebelum : 21 -> 47 -> 33 Operasi : tambahAkhir(head, 58) Sesudah : 21 -> 47 -> 33 -> 58 (kembalikan true) Sebelum : (kosong) Operasi : tambahAkhir(head, 21) Sesudah : 21 (kembalikan true)	

2. posisi

25 Poin

Quest	Hitung ada berapa orang yang masih mengantri di depan pasien bernomor `data`. Kalau nomor itu kebetulan berada paling depan, berarti tidak ada satupun orang di depannya berarti jawabannya 0. Kalau ia berada di urutan kedua, berarti ada 1 orang di depannya. Begitu seterusnya.
--------------	---

	Bisa juga terjadi nomor yang ditanyakan ternyata tidak ada di antrian, misalnya karena orangnya sudah dipanggil dan pergi. Dalam keadaan itu pertanyaannya tidak punya jawaban, jadi kembalikan -1 sebagai tanda "nomor itu tidak ada di sini".	
Parameter/ Variabel	head	Alamat orang paling depan. Melihat antrian tidak boleh mengubahnya. Boleh bernilai `nullptr`
	data	Nomor antrian yang sedang ditanyakan.
	return	Banyaknya orang yang berada di depan nomor itu, atau -1 bila nomornya tidak ada di dalam antrean
Gambaran	Antrean : 21 -> 47 -> 33 -> 58 <pre> 21 47 33 58 ^ ^ ^ ^ 0 1 2 3 <- berapa orang di depannya posisi(head, 21) // 0 — nomor 21 paling depan, tidak ada yang mendahului posisi(head, 47) // 1 — ada 1 orang di depannya, yaitu 21 posisi(head, 33) // 2 — ada 2 orang di depannya, yaitu 21 dan 47 posisi(head, 58) // 3 — paling belakang posisi(head, 99) // -1 — nomor 99 tidak ada di antrean ini posisi(nullptr, 21) // -1 — antriannya sedang kosong </pre>	

3. hapus

30 Poin

Quest	Keluarkan pasien bernomor `data` dari antrian, lalu buang datanya dari memori supaya tidak menumpuk sia-sia. Jadi ada dua pekerjaan di soal ini: - Menyambung ulang antriannya supaya tetap utuh dari depan sampai belakang - Membuang data orang yang sudah pergi dari memori Keduanya sama-sama dinilai.	
Parameter/ Variabel	head	Alamat orang paling depan, milik pemanggil. Bertanda `&`, sehingga pergantian orang terdepan ikut terasa oleh pemanggil
	data	nomor antrean orang yang membatalkan
	return	`true` bila memang ada orang yang dikeluarkan, `false` bila nomornya tidak ada di antrean
Gambaran	<pre> Sebelum : 21 -> 47 -> 33 -> 58 Operasi : hapus(head, 47) Sesudah : 21 -> 33 -> 58 </pre> (kembalian true)	

4. clear

15 Poin

Quest	Bubarkan seluruh antrean sekaligus, sampai benar-benar kosong seperti keadaan sebelum loket dibuka. Perhatikan baik-baik kata "dibubarkan". Memutus sambungannya saja tidak cukup. Anda memang bisa langsung membuat `head` bernilai `nullptr`, dan sekilas antreannya akan terlihat kosong. Tetapi seluruh data orangnya masih menumpuk di memori, dan sekarang tidak ada satupun yang bisa mencapainya lagi karena alamat orang terdepennya sudah hilang. Keadaan seperti itu disebut kebocoran memori, dan itu ikut dinilai. Jadi setiap orang harus benar-benar dibuang satu per satu lebih dulu, baru sesudah itu `head` dikosongkan.	
Parameter/ Variabel	head	Alamat orang paling depan, milik pemanggil. Bertanda `&`, sehingga antrian yang menjadi kosong ikut terasa oleh pemanggil
	return	tidak ada
Gambaran	Sebelum : 21 -> 47 -> 33 -> 58 -> 62 Operasi : clear(head) Sesudah : (kosong) `head` bernilai nullptr, panjang(head) bernilai 0, dan kelima orang sudah dibuang dari memori	

CATATAN & ATURAN

- Nomor antrian boleh negatif, boleh nol, dan boleh muncul lebih dari sekali.
- `head` bernilai `nullptr` berarti antreannya sedang KOSONG. Itu keadaan yang sah, bukan kesalahan, dan setiap fungsi harus menanganinya wajar.
- Parameter bertanda `&` berarti `head` milik pemanggil ikut berubah. Tanpa tanda `&`, fungsi hanya menerima SALINAN alamat orang paling depan salinan itu boleh digeser-geser untuk menelusuri, dan `head` milik pemanggil tetap aman.
- Setiap orang dibuat di memori dinamis dengan `new`, dan yang keluar dari antrian harus dibuang dengan `delete`.
- Tidak ada satupun fungsi yang mencetak ke layar.
- Signature fungsi dan bentuk `struct Node` adalah kontrak; isi fungsi, nama variabel, dan struktur kode di dalamnya sepenuhnya bebas.
- Anda boleh menambah fungsi bantu sendiri.
- Setiap soal dinilai sendiri-sendiri. Checker membangun antrian ujinya tanpa memakai fungsi Anda, jadi Soal 1 yang belum benar tidak ikut menjatuhkan nilai soal yang lain.